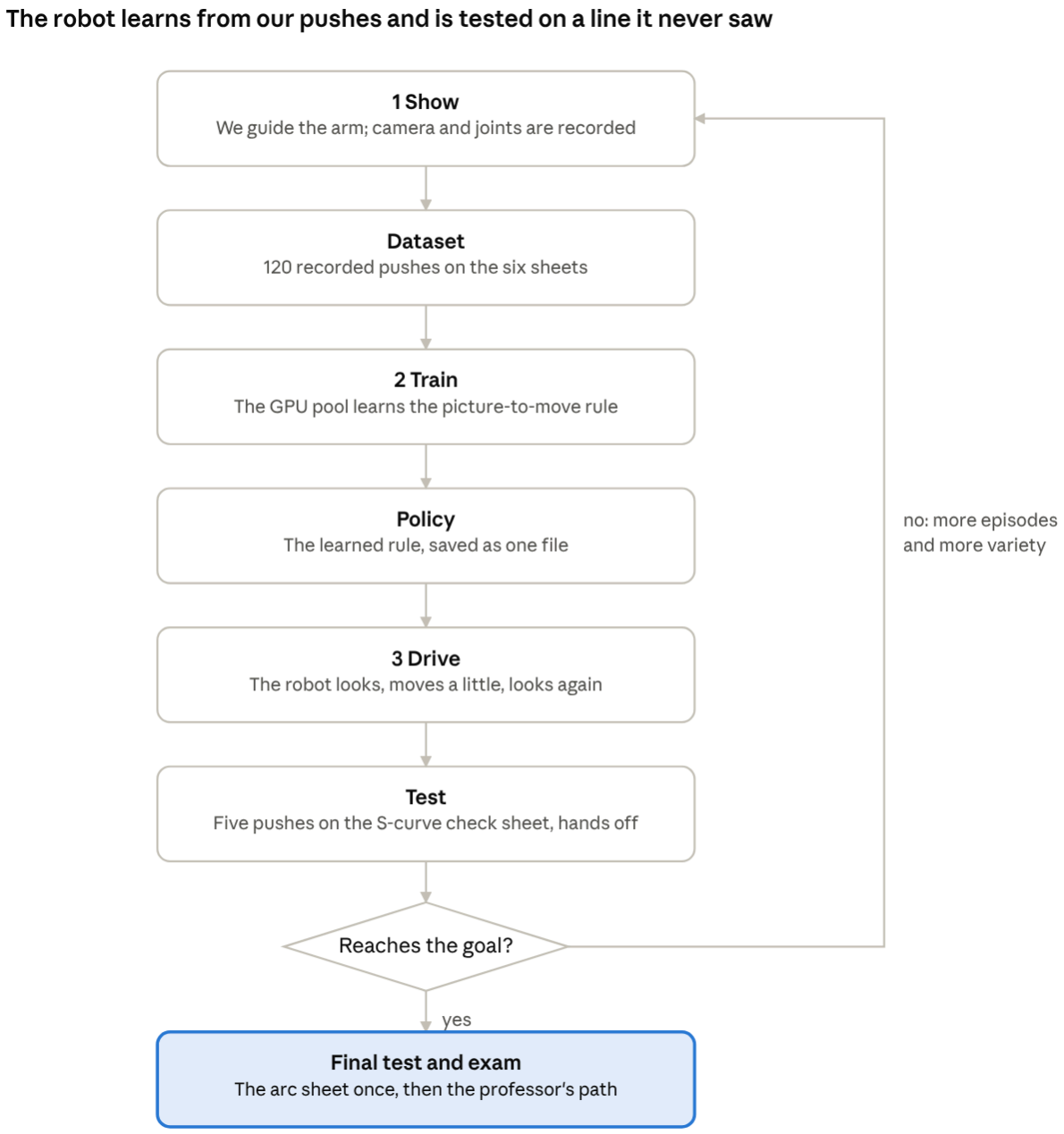
AI Project Guide

Oct 24, 2026 · @Emranul Hoque

What Team M is building for Project 6, how a robot learns to push a puck along a line by watching us do it, and what happens on exam day. Written for a first-time robot learner: every unfamiliar word is explained where it first appears and again in the glossary at the end.

What we are building, in one picture



from showing to the exam · 7 steps, 1 decision, 1 loop back

Our recorded pushes teach the robot a rule, the rule is tried on a sheet it never saw, and a failed try sends us back to record more. The words in the boxes are explained in the sections below.

What is on the desk

- The path sheet: a printed page with one black line. The puck starts at one end of the line; the other end is the goal. We have six sheets, from a straight line to a dashed self-crossing loop, and the professor keeps a seventh for the exam.
- The puck: a red wooden disc that slides over the sheet.
- The pusher: whatever sits at the tip of the arm and touches the puck. It should be round where it meets the puck, the bare steel shaft at the arm's wrist, which is what is left when the gripper is taken off, because a round contact can steer the puck around bends while a flat face cannot. We confirm this choice with the professor as question 4.
- The camera: the small camera on the robot's forearm, looking down at the pusher and the puck. It is the robot's only eye.
- The laptop: it runs our Python scripts and talks to the robot over the Ethernet cable. On exam day it is the computer that drives the robot.

The idea: teaching by showing

Imitation learning means the robot learns a skill by copying us, not by being programmed line by line. We move the arm, by hand or from the keyboard, so the pusher slides the puck along the black line, while a script records what the camera sees and where the arm is at every moment. From thousands of such moments a neural network, a program whose answers are shaped by a large pile of adjustable numbers, learns one rule: when the picture looks like this, move like that.

Why not simply program the path? Because the exam path is one we have never seen. A fixed programme only works for lines someone typed in; a learned rule has seen many lines, so a new one still looks familiar.

Think of teaching someone to drive. You do not hand them a map of one road. You let them watch and feel many trips, and afterwards they can handle a street they have never been on.

Three jobs, in this order:

1. Show: we push the puck ourselves, many times, on the six path sheets, while recording.
2. Train: a computer learns the picture-to-move rule from those recordings.

3. Drive: the robot runs the rule on its own, looking at the camera and moving a little at a time, until the puck sits in the goal.

Demonstrations: showing the robot

The professor said the hard part is not the training but getting a good dataset. A demonstration, also called an episode, is one complete push: the puck starts at the beginning of a sheet and ends in the goal, with a script recording the whole time.

There are two ways to move the arm while it records. Keyboard driving is our likely choice, because its pictures match what the robot sees later; the professor may settle it at meeting 1 (question 4), and we try both on the straight line first.

Way	How	Good	Bad
Hand-guiding	Learning mode switches the motors off and we steer the arm by hand, holding it at the elbow so our hands stay out of the camera's view	Natural and quick to record	The arm sags the moment the motors switch off, so hold it before enabling; height and tilt wander, and the camera moves while it records, so the pictures differ from the still, fixed-height ones the robot sees when it drives itself
Keyboard driving	The motors stay on and we move the tip a few millimetres per key press from the laptop	Hands never in the picture; the robot moves exactly as it will on exam day: same height, same tilt, still pictures	Slower; one push takes a minute or two

How one episode is made:

1. Tape a sheet flat on the table and put the puck at its start.
2. Run the start script: a fixed move, not the policy, brings the pusher to the start pose, at puck height and directly behind the puck, on the side away from the line ahead. Every recording and the exam start use this same arrangement, because the pusher's place behind the puck is how the policy knows which way to push.
3. Hand-guiding only: hold the arm at the elbow, then switch learning mode on; the arm sags when the motors go off.

4. Start the recording script.
5. Push the puck along the line into the goal, by hand or by key presses.
6. Stop the script. For hand-guiding, hold the arm and switch learning mode off. The script writes one folder: the pictures and a table with one row per tick.

What one tick holds, one row per picture, about nine per second:

Field	Example	Why it is needed
Time	12.437 s since start	Lines the picture up with the arm's position
Picture	One camera frame	What the robot will see when it drives itself
Joint angles	6 numbers, one angle per motor	Where the arm was
Tip position	x, y, z on the table and three tilt angles	Where the pusher was; a row further along the push is the answer the rule must learn (see What the model is)

How many episodes: 20 per sheet in the first round, 120 in all. Research systems that learn one skill from demonstrations typically use around 50, but they record faster, with more sensors, and are tested on the same layout they learned on. Our success on a new line depends more on variety and on how the step is defined than on the count. When the robot fails in a particular way, we record more episodes of that situation rather than more of everything.

Variety is what makes the unseen path work. Each episode should differ a little: the puck a centimetre off the start, the sheet shifted or turned slightly, daylight and lamp light. Push slowly and steadily anyway: a fast push blurs the pictures and lets the puck slide on after the pusher stops, which the labels cannot describe. Recovery episodes matter most: start the puck slightly off the line and bring it back, because that is exactly what the robot must do when it drifts.

What the model is

The model, which we call the policy, is the learned rule: a camera picture goes in, the next small step of the pusher comes out. The professor decided what goes in (the picture) and what comes out (a robot movement); only the middle is ours to build.

Inside, the policy is a neural network, a large pile of numbers that starts random. Training adjusts those numbers until, for each recorded picture, the network's answer matches the step we actually made next. Nobody writes a rule like "turn left when the line bends"; the rule emerges from the examples.

What "a small step" means is our choice, and it decides how much data we need. The forearm camera never sees the rest of the arm, and the sheet lies in a slightly different place in every episode, so a picture cannot tell the network where the arm is in absolute terms. The step must therefore be relative: move the tip this far forward and this far sideways from where it is now, with the pusher's height and tilt kept fixed at the values used in the recordings. That is two numbers for the step, plus a yes/no flag for being finished (see Pushing on its own): far easier to learn than six joint angles, and it keeps the pusher on the table.

Forward means away from the robot's base and sideways means around it. Those are the directions the camera's picture is lined up with, because the camera turns with the base, and the pusher always faces away from the base too. The script does the bookkeeping: it turns each recorded step by the base angle of that row before saving it as a label, and turns the policy's answer back before sending it to the robot. After a hand-guided episode the script also checks that height and tilt stayed within 5 mm and 5 degrees, and flags the episode if not.

Part	What it is for us	Where it comes from
Input	One camera picture, shrunk from 640 × 480 to 160 × 120 so the network stays small and fast	The recording; at run time the robot sends the picture over the cable. The current joint angles can be added as a second input later if the picture alone is not enough
Output	A step of the pusher tip: forward and sideways, in millimetres, about one centimetre long, plus a yes/no "finished" flag	In the recording, the tip position in the row where the tip had moved about one centimetre further on; "finished" is yes only in the last rows of an episode
Network	A small network of the standard kind used for pictures, with a few extra layers that turn what it sees into the three numbers	Written by us in PyTorch, the Python library for neural networks; trained on the GPU pool

The step is measured by distance, not time. Hand-guided recordings pause and speed up, so "one second later" would make the same picture mean a long step in one episode and no step at all in another. "One centimetre further along" means the same thing in every

episode.

We start with the simplest form of behaviour cloning: one picture in, one step out. More advanced forms, called ACT and Diffusion Policy, predict a short run of steps at once and move more smoothly, but they are harder to build and slower to run. We try them only if the simple one fails on the robot.

One check before any training: a black line is only a few pixels wide after shrinking. We look at a handful of shrunk pictures and make sure the line, and the gaps of the dashed sheet, are still visible.

Training: the computer learns the rule

Training turns the folders of episodes into one file of network weights, the numbers inside the network, called a checkpoint. It runs on the university's GPU pool, its graphics-card computers in room DC 1.07, reached by a remote log-in through the university network (SSH over the VPN); a first small model also trains on the laptop in under an hour. The GPU pool is only for training: the finished file is copied back to the laptop, and the laptop drives the robot.

The six sheets get three different jobs. We record on all six; the difference is which recordings the network is allowed to learn from.

Sheets	Job	Episodes
Straight line, meander, loop, dashed loop	Training: the network learns from these	80
S-curve (path_03)	Checking: its recordings tell us when to stop training; the network never learns from them	20
Arc (path_02)	Held back: touched only in our own final test, as our stand-in for the professor's unseen sheet	20

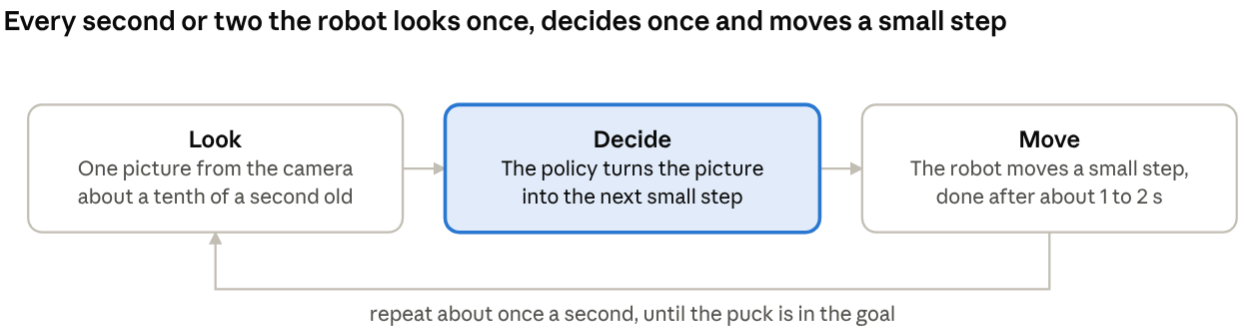
What happens, step by step:

1. Build the dataset: read every episode, shrink each picture, and pair it with the step the pusher made next, measured as the tip position about one centimetre further along. Scale all numbers to a similar range.
2. Split by sheet as in the table, never by random rows. Rows from one episode look almost identical, so a random split would check the network on pictures it has already seen and make the score look better than it is.

3. Train: the network sees pictures in batches, a few dozen at a time, guesses a step, is told how far off it was, and nudges its weights. One pass over all the data is an epoch; we run tens to hundreds.
4. Watch the loss, the average size of the mistake. The training script also prints the step error in millimetres with the scaling undone, and the "finished" output has its own error; both must fall on the check sheet's recordings, not just on the training ones. When they stop falling there, training is done; when they rise again, the network has begun to memorise.
5. Save the checkpoint with its settings in a dated folder under `04_runs/`, copy it to the laptop, and note which episodes went in, so any result in the report can be rebuilt.

A low loss is not yet success. A policy can match our recordings closely and still push the puck off the line, because one small error leads to a picture it has never seen. The only real test is the robot, so each training run ends with a few live pushes on a training sheet and on the S-curve check sheet. The arc goes on the table once, for the final test before the exam.

Pushing on its own



the control loop · look, decide, move, about once a second

Every round starts with a fresh picture, so a small error in one move is corrected by the next. That is what closed loop means, and it is why the policy never needs to know the whole path in advance.

The picture is quick: the camera gives about nine new pictures a second, so the one we read is at most about a tenth of a second old. The move is slow. When the script tells the robot to move, it must wait until the arm has arrived before it can take the next picture. We measured this on 24.10.26 at the 20 % speed cap, the limit we set in code on how fast the arm may move: even a tip shift too small to see costs 0.9 s, because the robot plans each move and speeds up and slows down gently.

How a one-centimetre tip step is sent	Time one step takes
Pose command, straight line in space	1.2 s
Pose command, joints interpolated	1.6 s
Joint angles from inverse kinematics, then a joint move	1.8 s
The same step at a 40 % speed cap	about a fifth less

So the robot gets one decision every one to two seconds, and a one-centimetre step costs little more than a one-millimetre one. With one-centimetre steps a 40 cm line takes about a minute. That is why the policy predicts steps of that size, and why the recordings are labelled by distance. The script sends each step as a straight-line pose command, the fastest of the three.

A picture-to-step policy cannot tell by itself that it is finished, and the forearm camera sees only a patch of the sheet, so the script needs its own rules for ending a run:

- Goal reached: the policy's third output, "finished", learned from the last rows of every episode; the script stops when it says yes twice in a row. A simple detector for the end of the line in the picture is the fallback.
- Lost: the puck has left the picture, or the step count passes the budget for that sheet, 1.5 times its length in centimetres.
- Unsafe: the next step would take the tip outside the workspace box, the sheet plus a 5 cm margin, defined in code. Such a step is refused and the run ends.
- Human: one of us presses the robot's emergency stop. The motors cut and the arm drops, so keep clear of it. A move already sent cannot be cancelled from the laptop; the script can only stop between moves, one more reason to keep steps small.

On exam day we measure the line with a tape before the run and type its length and the sheet's corners into the script, which sets the budget and the box. All four rules are in the code, and the emergency stop has been tried, before the first push with nobody touching the arm.

Reaching the goal and being judged

On exam day the professor lays down a sheet we have never recorded and places the puck at its start. A fixed scripted move brings the pusher to the start pose behind the puck, and from then on the robot must push it to the goal on its own: nobody touches the arm or the keyboard once the policy starts. That is the whole test of Project 6.

How the run is scored is not announced yet. The likely measures are whether the puck reaches the goal, how far it strays from the line on the way, and how long it takes. We have asked the professor exactly this as question 8 for the meeting on 09.11.26, including what "in the goal" means; our working rule is the puck resting over the end of the line, within about one puck width.

Our own exam, run before the real one: the held-back arc sheet, five pushes, and a note of how many reached the goal and the largest distance the puck strayed from the line. The same five pushes on a training sheet tell us whether a failure is a weak policy or a line it cannot generalise to.

What we hand in, and when:

Deliverable	Due	What it contains
Submission on Moodle	10.01.27, 23:59	Code, the trained checkpoint, the recordings and the evaluation results; the exact list is question 7 for the professor
Documentation	10.01.27, 23:59, inside the Moodle submission	About 7 pages: the task, how we recorded, the model, the results on seen and unseen lines, what failed and why
Presentation	11.01.27	15 minutes as a team, likely with a live run; whether a live demo is expected is question 11, for the meeting on 23.11.26

Each of us must have an identifiable share of the work, since at least half of the grade is individual. Commits, episode folders and report sections should carry the name of whoever made them.

Why it can fail and what we do about it

Most first attempts fail, and almost always for one of the reasons below. Knowing them in advance turns a bad day at the robot into a known fix.

What goes wrong	What you see	The fix
Drift: one small error puts the puck somewhere we never showed, and the policy has no idea what to do there	The push starts well, then wanders off and never comes back	Record recovery episodes: start the puck slightly off the line and bring it back. Later, let the policy drive and, between two moves, press the key that hands control to the keyboard; finish the push from the keyboard and record it, keeping only the rows we drove
Too few or too similar episodes	Works on a training sheet, fails on any other	More episodes with more variety: shifted sheets, both turn directions, the dashed loop, both lighting conditions
Our hand in the picture	Fine in training, lost on the robot	Hold the arm at the elbow or drive it from the keyboard, and look at the first recording picture by picture
Camera changed between recording and exam	Policy behaves as if blind	Never touch the focus ring again; keep the same mount, speed cap and table height; take a reference picture before each session and compare its sharpness
Steps too coarse for a bend	Puck overshoots the corner	Smaller steps, or a policy that outputs a short run of steps at once
No end to the run	Robot keeps pushing past the end of the line	Train the "finished" output, set the step budget, and test both on a training sheet first

What goes wrong	What you see	The fix
Puck tips, slips or slides along the pusher	Episode ruined, or the puck drifts sideways on every curve	A round pusher at puck height (the bare steel shaft at the wrist), never a flat face: with the pusher's orientation fixed, only a round contact can steer the puck round a bend. Lower speed and a clean sheet taped flat
Policy memorised the sheets	Very low loss on the training sheets, poor loss on the check sheet	Stop training earlier, shrink the network, add small random shifts and brightness changes to the training pictures
Robot safety	Arm heads for the table edge or a person	Speed cap at 20 %, the workspace box in code, the step budget, a hand near the emergency stop

Where we are today and what comes next

As of 24.10.26 the robot, the camera and the laptop all work together from code, which is the whole of phase 1 in `work_plan.pdf`. Nothing has been recorded yet.

Done:

- NiryoStudio 1.16.3, the robot's own desktop app, and a Python 3.11 environment with pyniryo (the robot library) and OpenCV (the picture library) on the laptop.
- The robot answers on the cable, calibrates (finds its zero positions after power-on), and moves from a script: home pose and a 17-degree base turn at 20 % speed.
- The forearm camera is plugged into the robot, streams 640 × 480 pictures, about nine new ones per second, and is focused, with a reference picture saved to compare against later.
- Read and move timings measured and written to `config.py`.
- Course materials, the six path sheets, the folder structure and the list of questions for the professor.

Next, in order:

1. Meeting 1 on 26.10.26: ask questions 1 (the original path-sheet files), 3 (may we print our own sheets), 4 (how to demonstrate and which pusher), 6 (GPU pool access) and 7 (what exactly to submit); the full list is in `05_meetings/requests_to_professor.md`.
2. Workspace: tape the straight-line sheet flat, fit the pusher, define the start pose (tip at puck height, directly behind the puck at the start of the line), the pushing height and the workspace box around the sheet.
3. Recording script: one folder per episode, one row per picture, about nine per second, with the joint angles and tip position, named `path_01_straight_line_episode_001` and so on. Try hand-guiding and keyboard driving, and keep the one whose pictures are cleaner, unless the professor settles it at meeting 1.
4. Ten episodes on the straight line, then look at them picture by picture, shrunk to 160 × 120, before recording more.
5. A first behaviour-cloning model on the laptop, the four stop rules in code, the emergency stop tried, and then the first push with nobody touching the arm.
6. All six sheets, the GPU pool, the check-sheet tests, the one final test on the arc, then the documentation and the presentation.

Glossary

Word	What it means here
Path sheet	A printed page with one black line; one end is the start, the other the goal
Puck	The red wooden disc the robot pushes
Pusher	The tool at the tip of the arm that touches the puck; round where it meets the puck
Start pose	The pusher at puck height, directly behind the puck at the start of the line; a fixed scripted move takes the arm there
Imitation learning	Teaching the robot by showing it, then letting a network copy the examples
Behaviour cloning	Training a network to copy our recorded moves directly; our first version is its simplest form, one picture in, one step out, trained like an ordinary picture-recognition network except that it predicts numbers rather than names

Word	What it means here
Policy	The learned rule from picture to step; the thing we train and the laptop runs
Step	The policy's answer: move the pusher tip about one centimetre, this far forward and this far sideways
Episode	One complete recorded push from the start of a sheet to its goal
Tick	One row of the recording: a picture, the joint angles, the tip position and the time, about nine per second, one per picture
Dataset	All episodes, shrunk and paired so the network can learn from them
Training	Adjusting the network's numbers until its steps match ours
Weights	The numbers inside the network; training changes them, the checkpoint stores them
Epoch	One pass over the whole dataset during training
Loss	The average size of the network's mistake; lower is better
Checkpoint	The file of trained weights that the laptop loads to drive the robot
Closed loop	Look, move, look again: each move is chosen from a fresh picture
Check sheet	A sheet whose recordings are never learned from, used only to tell when training should stop and for live tests after each training run
Held-back sheet	A sheet used only in our own final test, as a stand-in for the professor's unseen one
Generalise	Work on a line the policy has never seen
Drift	The robot ends up in a situation that was not in the recordings and goes wrong from there
Learning mode	The robot's motors switch off so the arm can be moved by hand; it sags, so hold it

Word	What it means here
Calibration	What the robot does after power-on to find the zero position of every joint; required before it moves
Speed cap	The limit we set in code on how fast the arm may move, 20 % of its maximum
Joints	The six motor angles, in radians, that together describe the arm's shape
Tip position	Where the pusher is: x, y, z on the table and three tilt angles
Workspace box	The sheet plus a 5 cm margin, defined in code; a step that would leave it is refused
Emergency stop	The robot's physical stop button; it cuts the motors and the arm drops
GPU pool	The university's graphics-card computers in DC 1.07, reached over SSH, where training runs